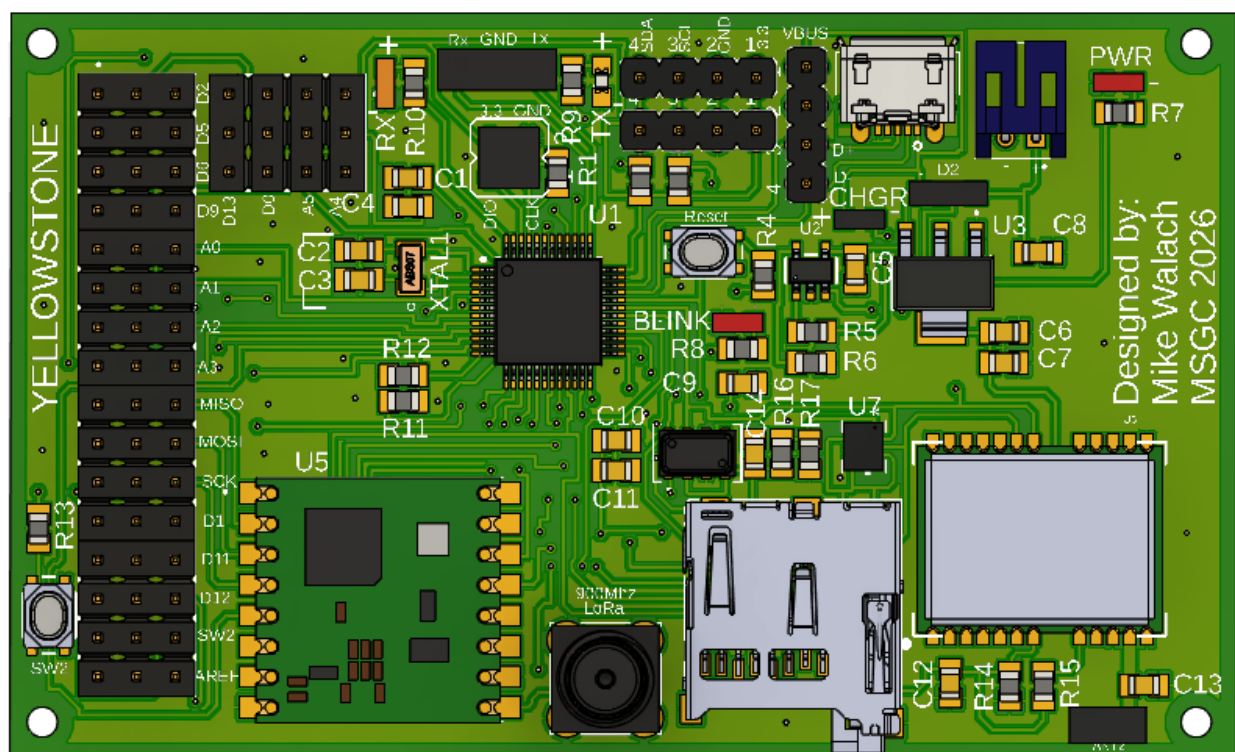


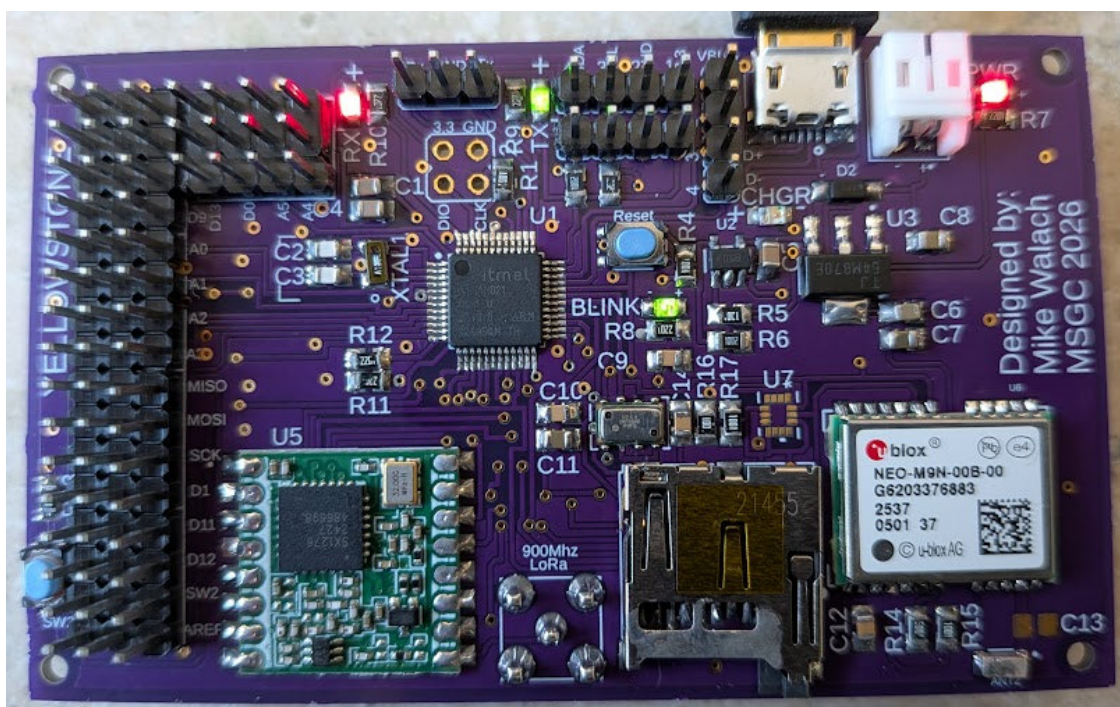


YELLOWSTONE

Your Experimental Low Orbit Wireless nanoSatellite for Telemetry & Operations in Near-space Environments

Designed by: Mike Walach 2026 BOREALIS





Overview:	3
PCB Identification	4
GPS	4
Quick Reference Guide	5
User I/O Ports	6
Serial I/O Header (Tx Rx)	6
I2C Header	7
USB Header	7
Programming Header	7
Electrical Schematics	8
Microcontroller	8
Power	9
Indicators	10
SD card, Serial Tx/Rx indicators, Voltage monitor and Tx/Rx I/o and USB breakout	11
I/O Headers	11
Pressure Sensor	12
User input switch	12
GPS	13
LoRa	13
Micro USB and Program Header	14
Accelerometer and Gyroscope	14

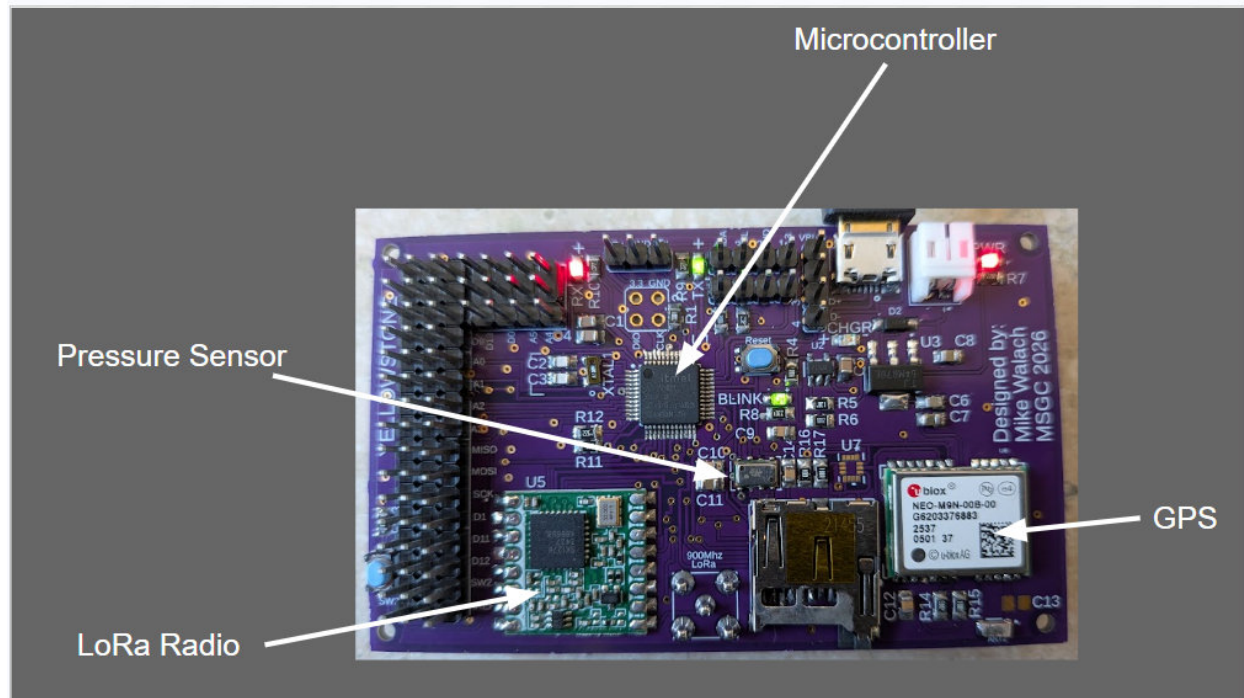
I2C Headers	15
Bill of Materials	15
Initial run-up and testing/Troubleshooting.....	17
Primary tests.....	17
Installing the bootloader.....	18
Setting the bootloader fuses (optional)	25
Next steps	26
Hardware Tests and Arduino Code Samples	26
BLINK.....	26
Blink Sample Test Code	27
SD	27
SD Card Sample Test Code	28
Pressure Sensor (Altitude).....	30
Pressure Sensor Sample Test Code	30
GPS.....	32
GPS Sample Test Code	33
Gyro	34
Gyro Sample Test Code.....	35
LoRa.....	35
LoRa Test Transmit code (Tx).....	35
LoRa Test Receive code (Rx)	37
Setting up YELLOWSTONE for flight operations	39
Ground Side Setup	39
Mission Control PC Software	39
Airborne Side Setup.....	40

Overview:

The YELLOWSTONE board is Arduino compatible and designed to meet the 1U CubeSat size requirements (less than 100x100x100mm) for a HAB (High Altitude Balloon) or UAS (Unmanned Aerial System) platforms. YELLOWSTONE provides the ability for data logging as well as LoRa data transmission. On-board sensors include GPS, Pressure, Temperature, Accelerometer, and Gyro. Headers (0.100" Spacing) with power and ground are provided and include serial I2C, SPI, and Tx, Rx. Analog and Digital I/O are also provided at the headers. Data can be logged to an on-board micro SD card. One test LED is located on D13 (same as the Arduino "Built-in" so the Arduino "Blink" sample code will work) and one input switch (SW2) is included for user input or test. This platform can be used in a variety of settings and allows for users to integrate

their own sensors or simply use the on-board sensors. YELLOWSTONE is NOT designed for orbital flight, rather it is a HAB testbed intended to simulate orbital flight at the edge of space (100,000 feet MSL [Mean Sea Level]). Two YELLOWSTONE boards can be used together to form an airborne and ground station pair when using the built-in LoRa radio.

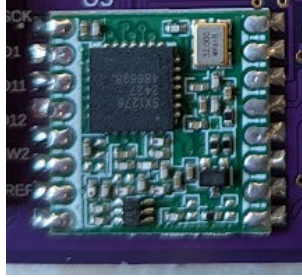
PCB Identification



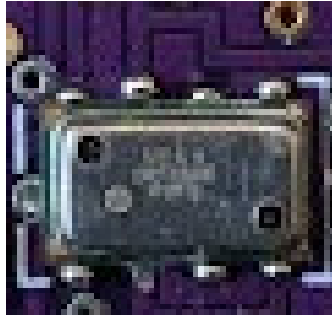
GPS



LoRa Module

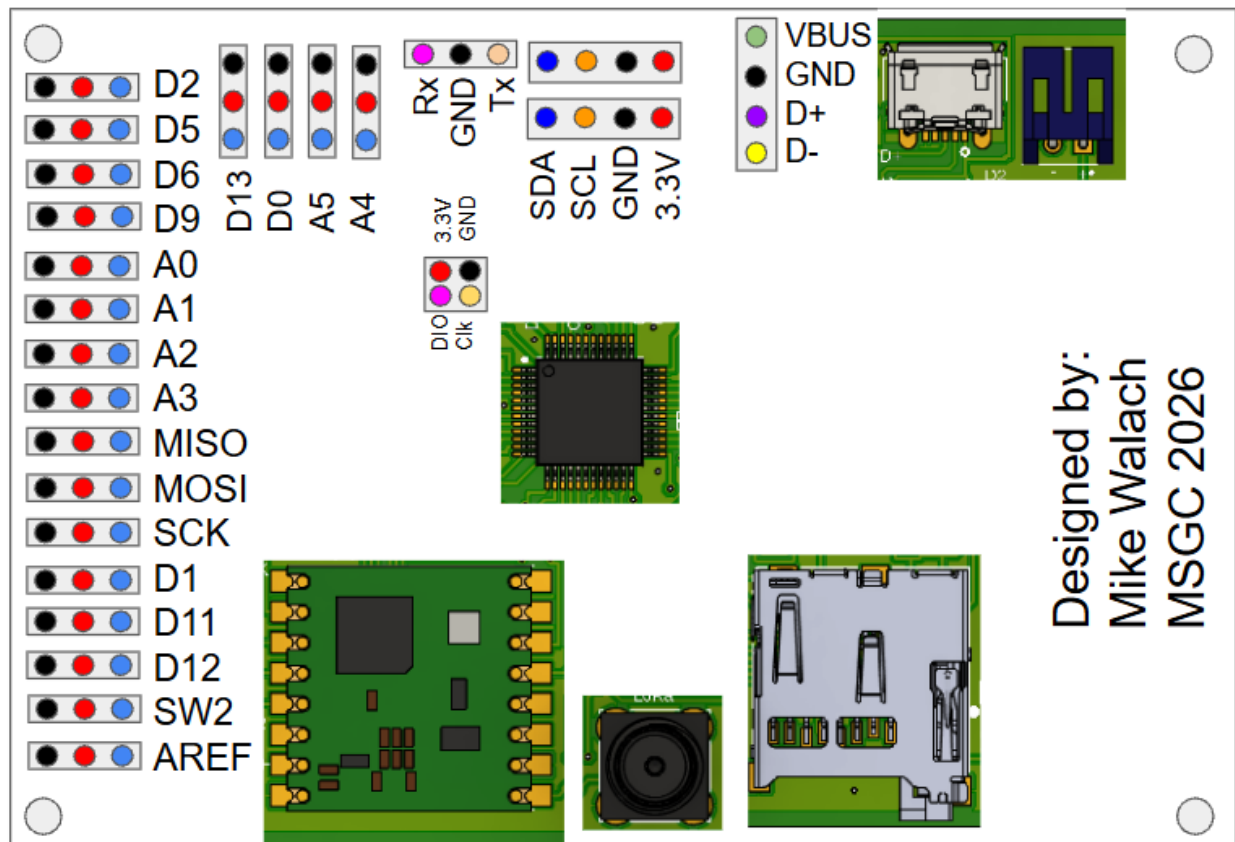


Pressure Sensor

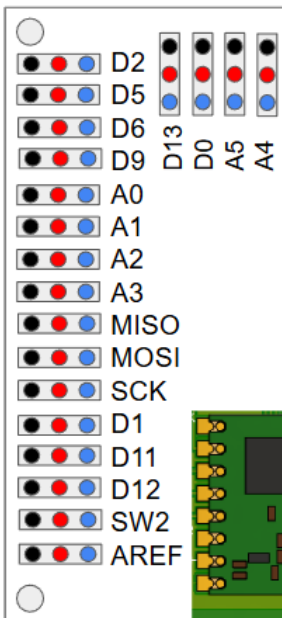


Quick Reference Guide

(Black = GND and Red =3.3V)

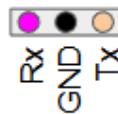


User I/O Ports



The blue dots on the I/O headers (left side of board and top left) are internally connected to their corresponding microcontroller pins, with the exception of SW2. The blue dot labeled SW2 is connected only to the pushbutton circuit, which is pulled high when the button is not pressed and pulled low when the button is pressed. To use SW2, it must be externally connected to a microcontroller input pin. The simplest method is to place a jumper between the SW2 blue dot and the D11 blue dot, allowing D11 to read the button state directly. Alternatively, SW2 may be connected to any available input pin. The black dots represent ground and the red dots represent 3.3VDC.

Serial I/O Header (Tx Rx)



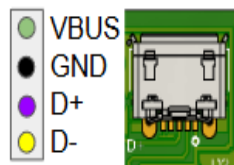
The three pin header at the top of the board just left of center, is the serial Tx and Rx lines with a ground pin in the center. Tx and Rx leds are located on either side of this header. The serial header allows connectivity of two YELLOWSTONE PCBs or other serial devices.

I²C Header



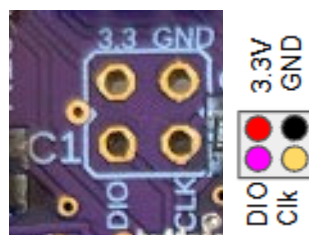
Two I²C headers are located at the top center of the PCB, each providing SDA, SCL, GND, and 3.3V connections. On both headers, SDA is indicated by blue dots, SCL by orange dots, GND by black dots, and 3.3V by red dots. These two headers are wired in parallel, allowing multiple I²C devices to be connected.

USB Header



A separate header, located to the left of the USB connector, provides access to the USB lines: D+, D-, VBUS (5V), and GND. **Use caution when working with VBUS, as it supplies 5V (not 3.3V)!** Shorting VBUS may damage the connected device, such as a laptop or other power source.

Programming Header

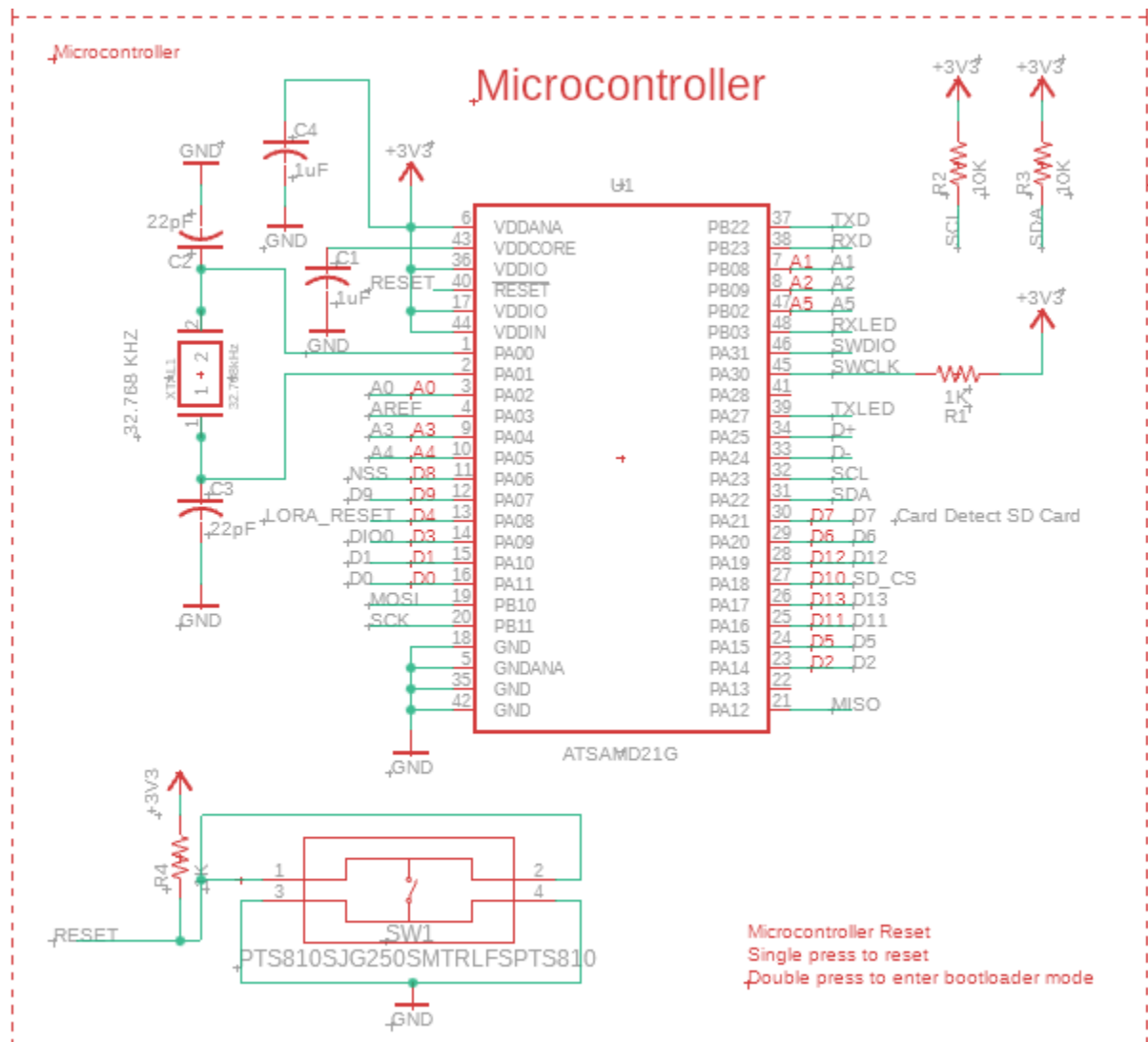


No header is soldered to the programming port pads. This port is used only for loading a bootloader onto the PCB. The pad assignments are as follows: top left = 3.3V, top right = GND, bottom left = DIO, and bottom right = SCLK. Access to this port is generally only required during initial manufacturing or when replacing the microcontroller, and it is not typically intended for end-user use.

Electrical Schematics

If you would like to fabricate your own YELLOWSTONE PCB, or, if you would like to modify or improve upon the design, you can download the Fusion360 files [here](#).

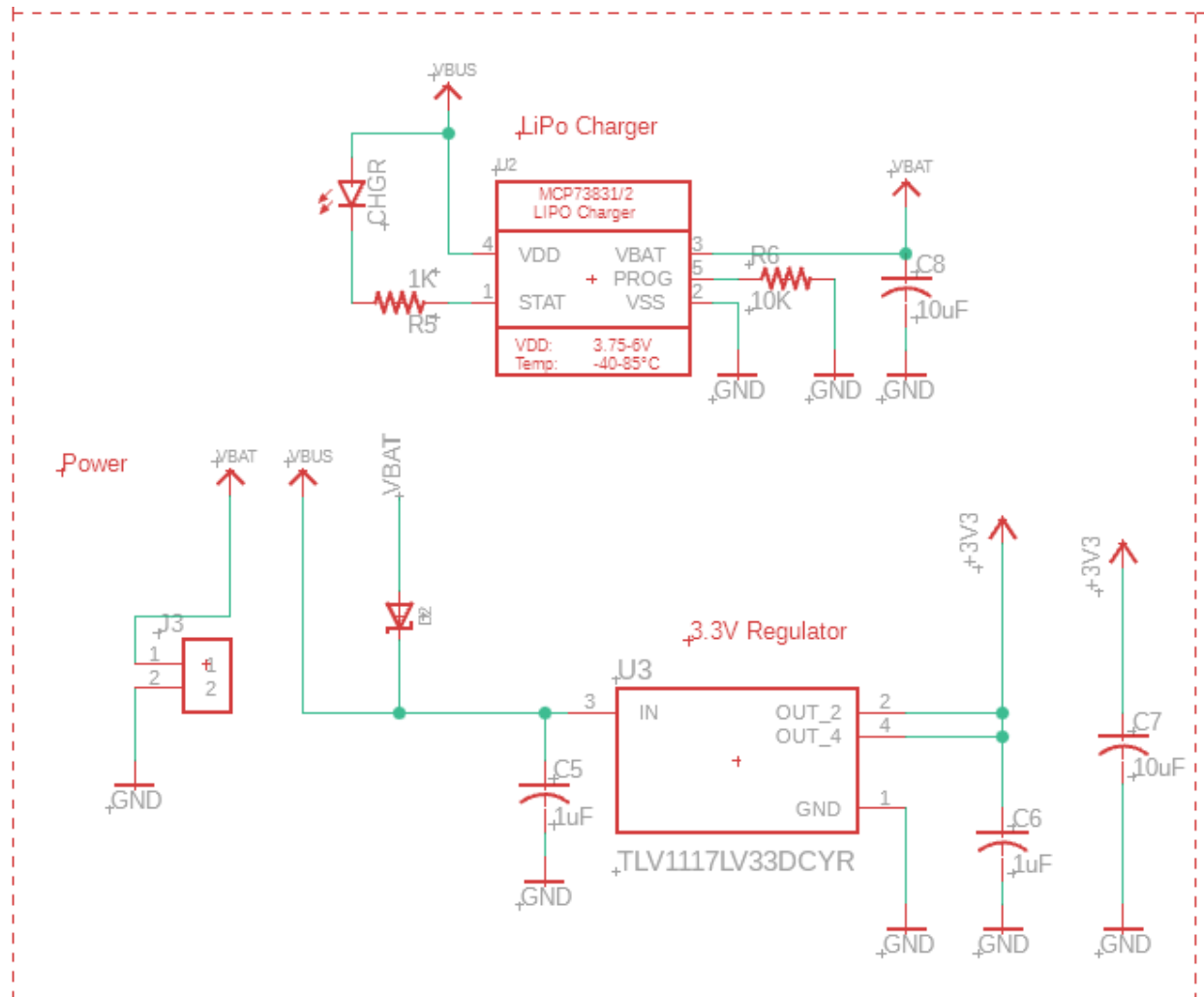
Microcontroller



The SAMD21G microcontroller is the heart of the YELLOWSTONE pcb. This microcontroller was selected because it is the heart of most MSGC designs and is robust, fast, and has many features that make it versatile enough to handle most HAB and UAS data collection tasks. An external 32.768khz crystal supplies the timing for the microcontroller. A reset switch (SW1) is provided on board for resetting the micro (single

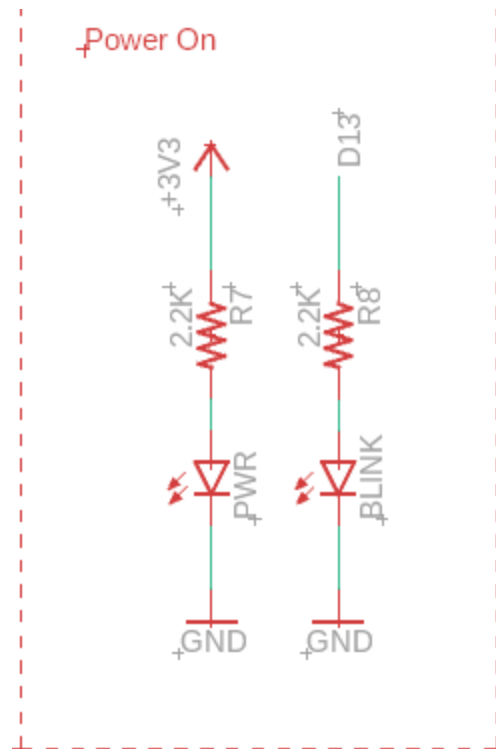
tap) and placing the micro into bootloader mode (double tap). 10K I2C pullups (R2 and R3) are provided.

Power



Power regulation is provided by a 1amp 3.3V dc regulator (U3). A diode (D1) prevents backflow from a battery to a USB port allowing the YELLOWSTONE to be powered by a battery and USB at the same time. A LiPo charge controller is built into the YELLOWSTONE board (U2) allowing for easy charging of flight batteries. As long as a battery is connected and a USB cord is supplying power, the battery will get charged. Note, there is not on/off switch on the YELLOWSTONE. This is an intentional design consideration to eliminate points of failure. Therefore if a battery is plugged in, the YELLOWSTONE is powered on.

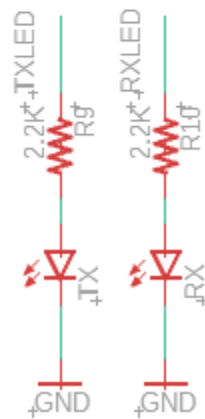
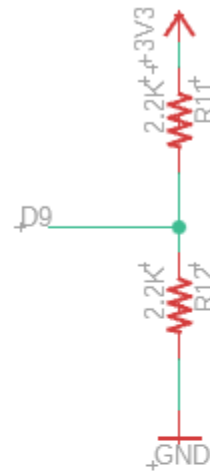
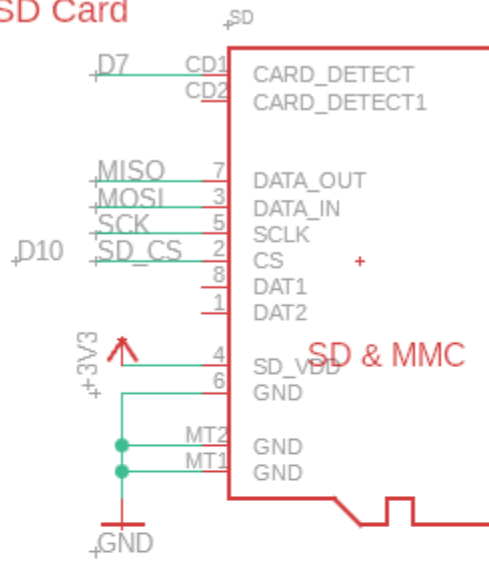
Indicators



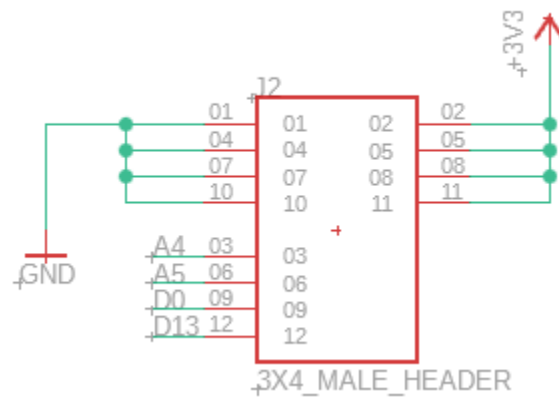
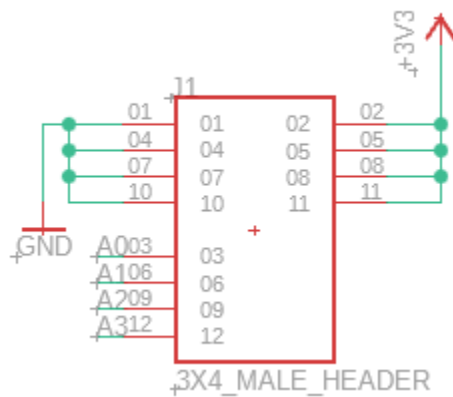
The PWR LED should turn on anytime the YELLOWSTONE is powered on, either with a battery or via USB. The “BLINK” LED is connected to D13. This allows the user to use this pin as a test or indicator output. Also, the default “Blink” sample code provided with Arduino will work on this PCB. This is a good way to confirm the YELLOWSTONE is working.

SD card, Serial Tx/Rx indicators, Voltage monitor and Tx/Rx I/o and USB breakout

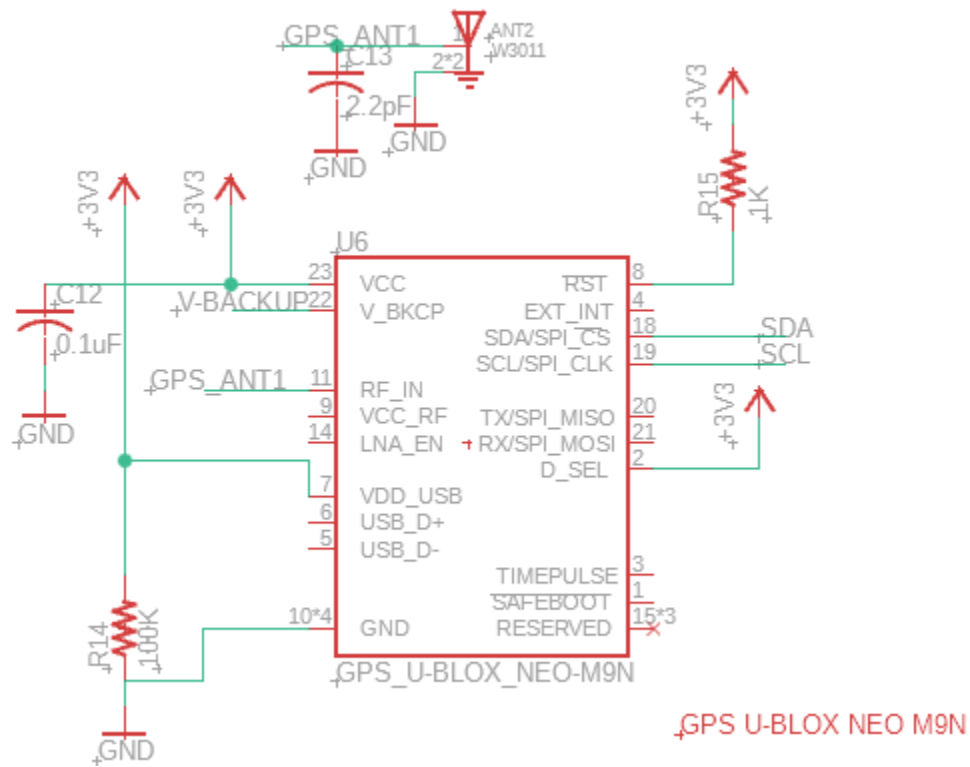
SD Card



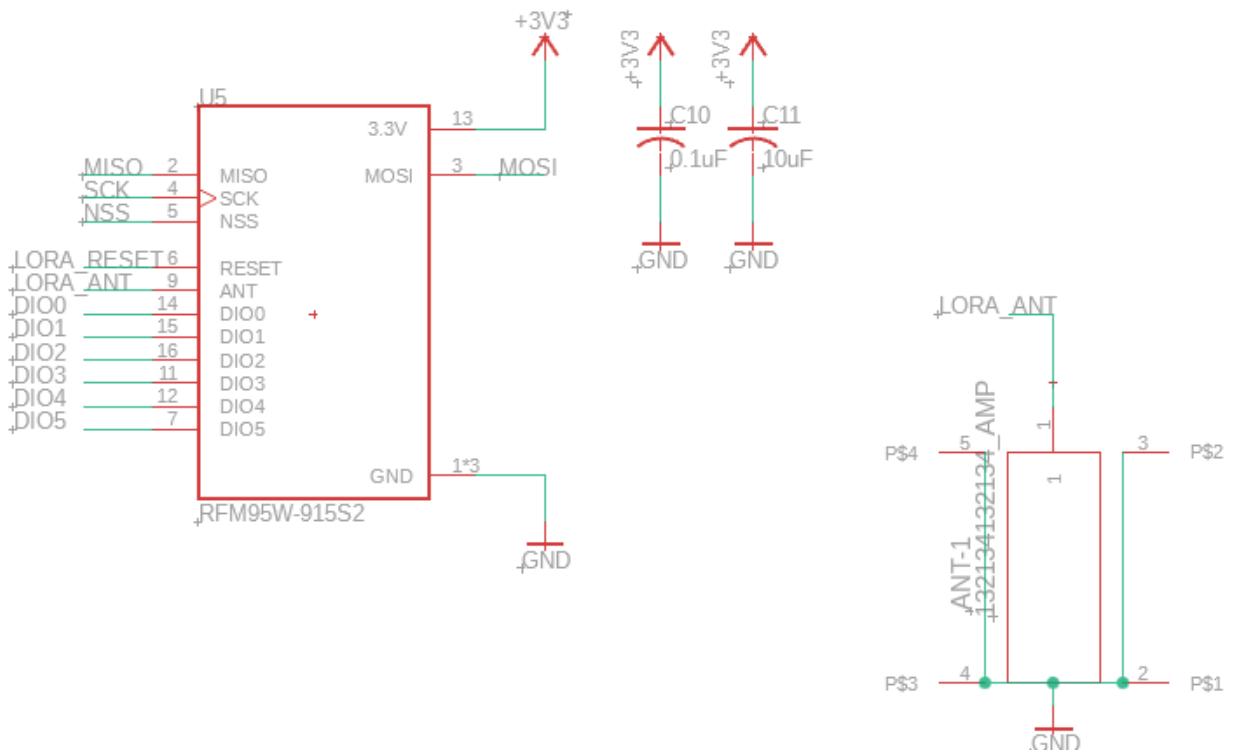
I/O Headers



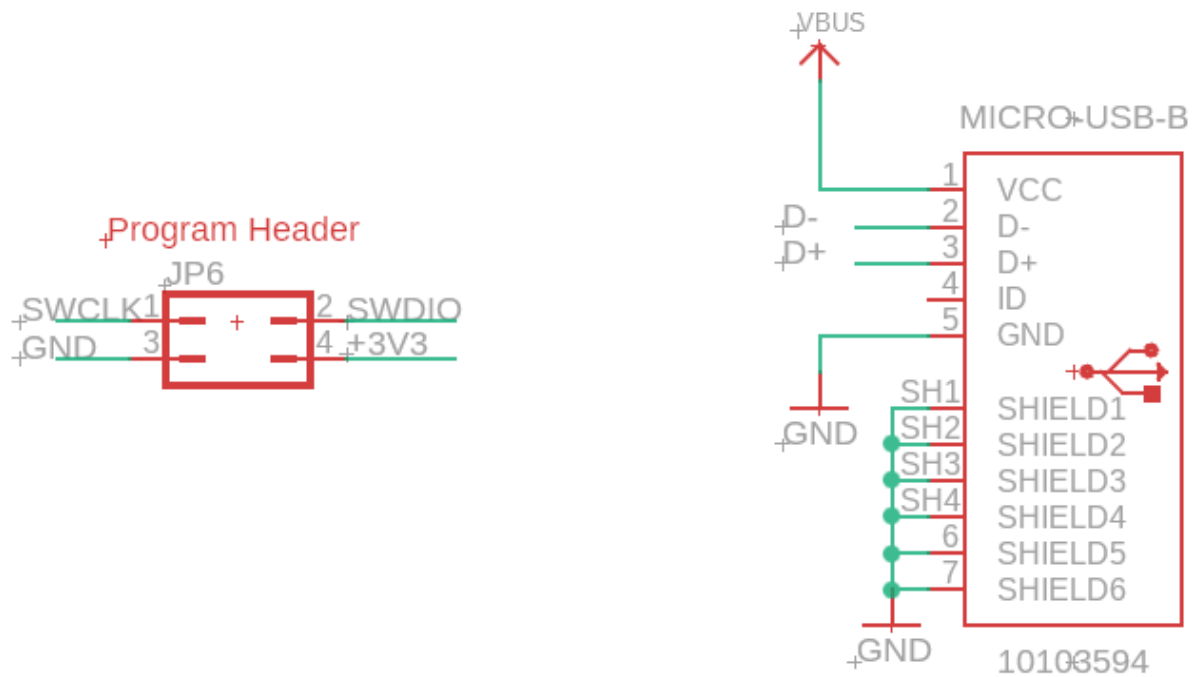
GPS



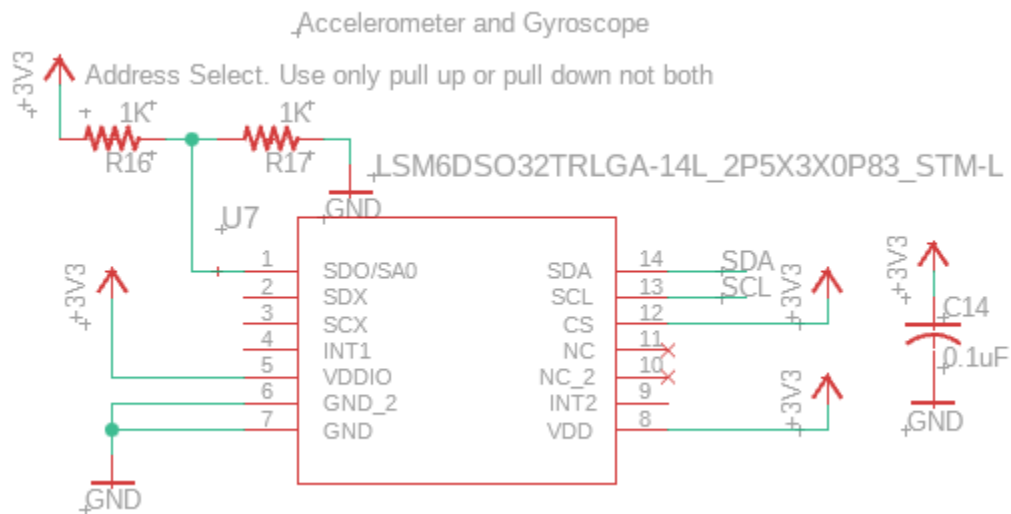
LoRa



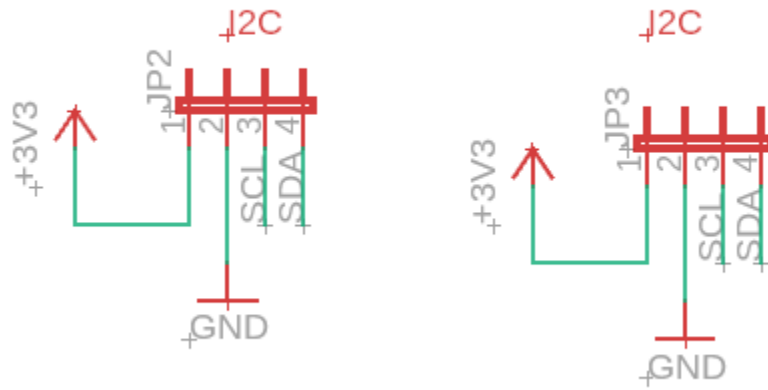
Micro USB and Program Header



Accelerometer and Gyroscope



I2C Headers



Bill of Materials

A complete parts list with vendor links is provided in the link below.

Link to: [Yellowstone V2 Buildsheet](#)

This chart is a good reference to identify the value or component type for any of the parts on the YELLOWSTONE PCB. It is also used when building the PCB.

Part	Label
0.1uF	C10, C12, C14, C9
1uF	C1, C4, C5, C6
10uF	C11, C7, C8
22pF	C2, C3
2.2pF	C13
1K	R1, R13, R15, R16, R17, R4, R5
10K	R2, R3, R6
100K	R14
2.2K	R10, R11, R12, R7, R8, R9
Switch Tactile	SW1, SW2

3.3V Linear Regulator	U3
Li-po Charger	U2
LoRa Module	U5
Accelerometer and Gyroscope	U7
Pressure Sensor	U4
Antenna Connector	900 Mhz LoRa
Micro SD Card Holder	SD
Micro USB-B	No Label
32.768KHZ Crystal	XTAL
GPS Chip Antenna	ANT2
Schottky Diode 20V 1 Amp	D2
3x4 male header A0,A1,A2,A3 & 3.3v/GND	0.100" Header
3x4 Male Header A4,A5,D8,D13 & 3.3v/GND	0.100" Header
2pin Connector-Battery input	JST 2-pin White
6 pin male header (optional) Programming Header	No don't solder header
4 pin male header I2C	0.100" Header
4 pin male header Vbus/Gnd/D+/D- (USB Breakout)	0.100" Header
4 pin male header Tx/Gnd/Rx	0.100" Header
LED RED CLEAR CHIP SMD	PWR
LED GREEN CLEAR CHIP SMD	BLINK
LED ORANGE CLEAR CHIP SMD	CHGR
LED GREEN CLEAR CHIP SMD	Tx
LED RED CLEAR CHIP SMD	Rx
Microcontroller	U1
M9 STANDARD PRECISION GNSS MOD	U6
LoRa Module	U5

Initial run-up and testing/Troubleshooting

NOTE: If you have received a new YELLOWSTONE PCB from the BOREALIS lab, you do not need to run these initial tests as we will have already done so. However, if you want the experience of testing your board's systems and/or learn how we test new boards after fabrication, this section may be helpful to you.

The YELLOWSTONE board should be brought online in a careful and methodical manner to prevent damage. This process only needs to be performed once, after initial fabrication, to verify all board functions before it is integrated into a larger system. However, if you are having trouble with your YELLOWSTONE board, this can serve as a good troubleshooting guide to test the basic systems and try to narrow down the potential fault in your board.

Primary tests

Using a VOM set to resistance or continuity, measure between power and ground. There should be measurable resistance present. As long as the meter does not indicate a short circuit, it is safe to connect a battery.

The YELLOWSTONE is designed to operate from a 3.7V lithium battery via a JST connector.



There is no power switch on the YELLOWSTONE PCB. This is an intentional design choice to eliminate potential points of failure during flight. When connected to a battery, a power LED should illuminate. The power LED is located on the top-right side of the YELLOWSTONE PCB..



If the power LED illuminates, this is a good indication that the board is receiving power; however, you must verify that the voltage regulator is providing a clean 3.3 V output. Using a meter set to DC voltage, measure between power and ground to confirm a

reading of 3.3 V (± 0.1 V). The programming header (without pins), shown below, provides a convenient location to place the meter leads and check for 3.3 VDC.



If the YELLOWSTONE PCB passes the voltage test, connect the micro USB connector to a PC or another 5 V power source (ensure the battery remains connected). This will test the charging circuit. If the charge LED illuminates, the LiPo battery is charging and the charging circuit is functioning properly. The charge LED will turn off once the battery is fully charged.

Installing the bootloader

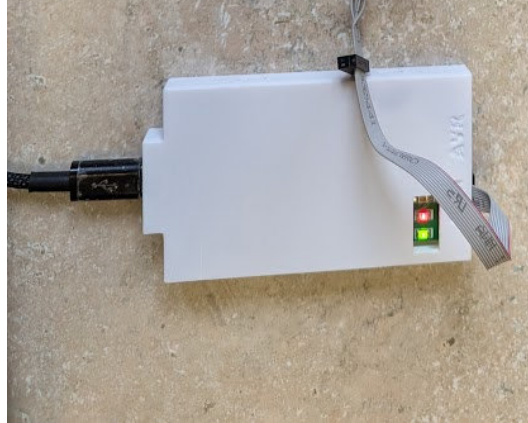
Note: If you have received a new YELLOWSTONE PCB from the BOREALIS lab, we will have already installed a bootloader for you and you do NOT need to do these steps! These are provided here for anyone building a new YELLOWSTONE PCB from scratch, or, for those who think their bootloader may be corrupted or needs updating.

If the YELLOWSTONE board passes all of the tests so far, it is time to install the bootloader. We like to use the Adafruit Adalogger bootloader. You can download a copy here: [Bootloader](#)

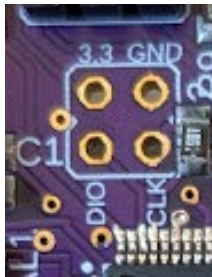
The bootloader needs to be installed using a [programmer](#) and [Microchip Studio](#). (Note: There are other ways to install a bootloader using other software and hardware. We are showing you how we do it in our lab, however, this is not the only way to accomplish this task).



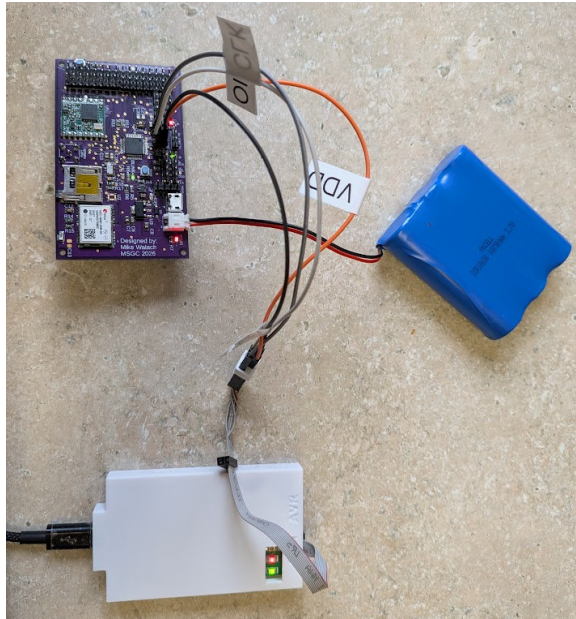
Once Microchip studio is downloaded and installed on your computer, connect your computer to your ICE programmer via USB as shown below:



Connect your ICE programmer to your YELLOWSTONE board as shown below. You will need to connect to SDIO, SCLK, 3.3V, GND. We use male to male jumper wires and carefully hold them in place while we program the board. We don't bother soldering header pins to the programming pads because you typically only need to program the bootloader on the board once after initial fabrication then never again. However, if you prefer to add a header to make this part easier, there is no reason not to. The programming header on the YELLOWSTONE PCB looks like this:



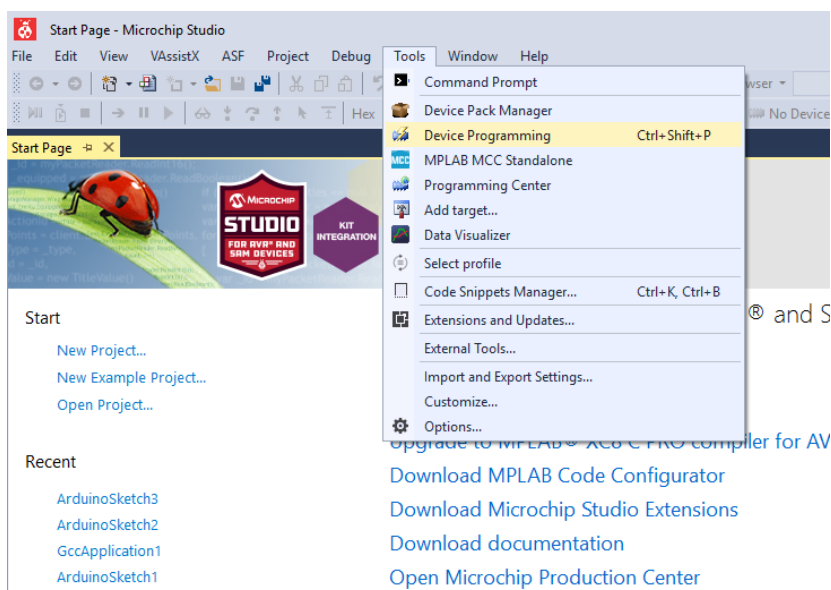
Note the location of the 3.3V pad top left, GND top right, SDIO bottom left and SCLK bottom right. Make sure that these correspond to the same connections on your programmer. See the documentation that came with your programmer to be sure you have these correct. Use caution not to short the power and ground pins at any point in this process or you could destroy the voltage regulator on the YELLOWSTONE PCB.



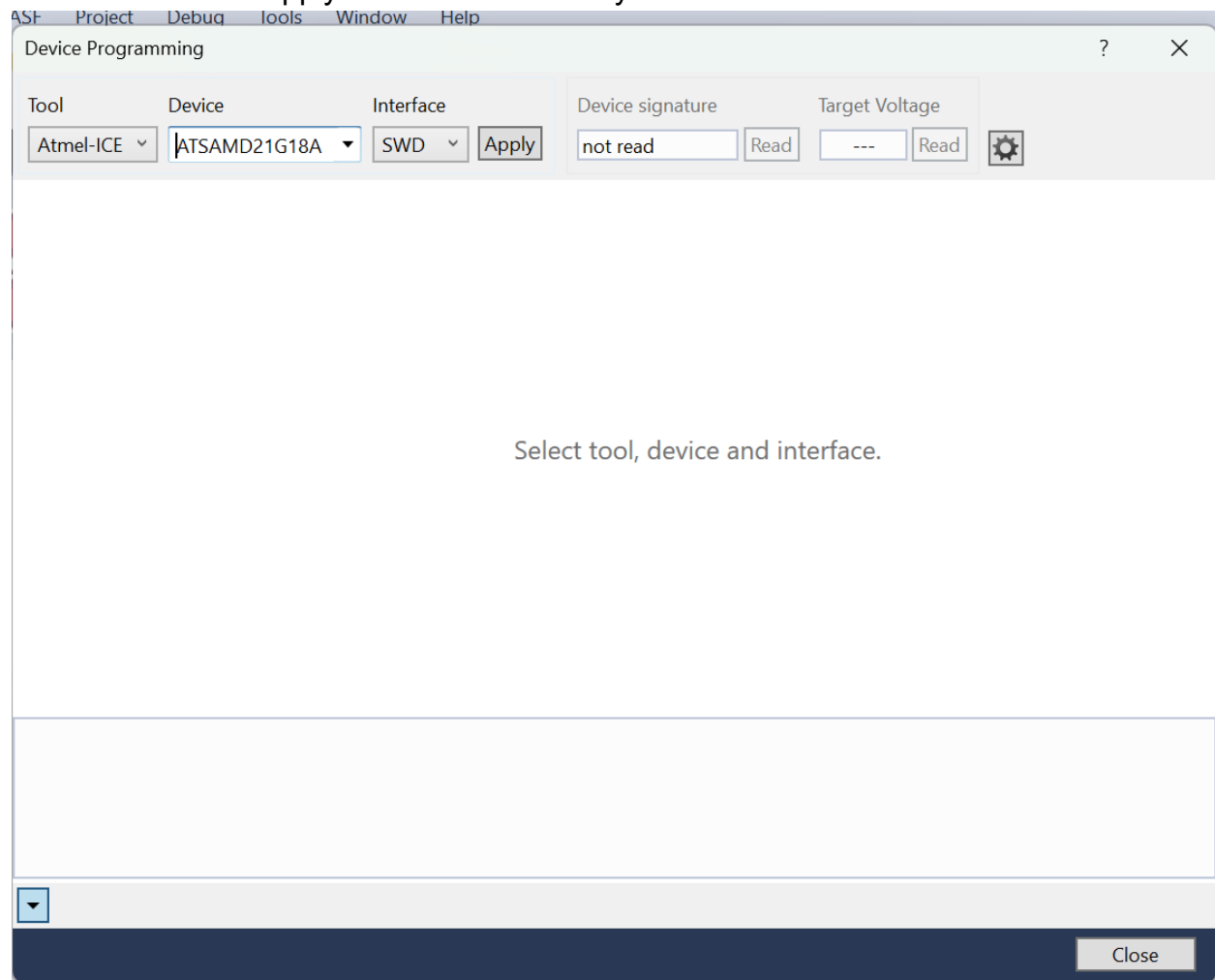
Connect a battery to your YELLOWSTONE board using the white JST connector.



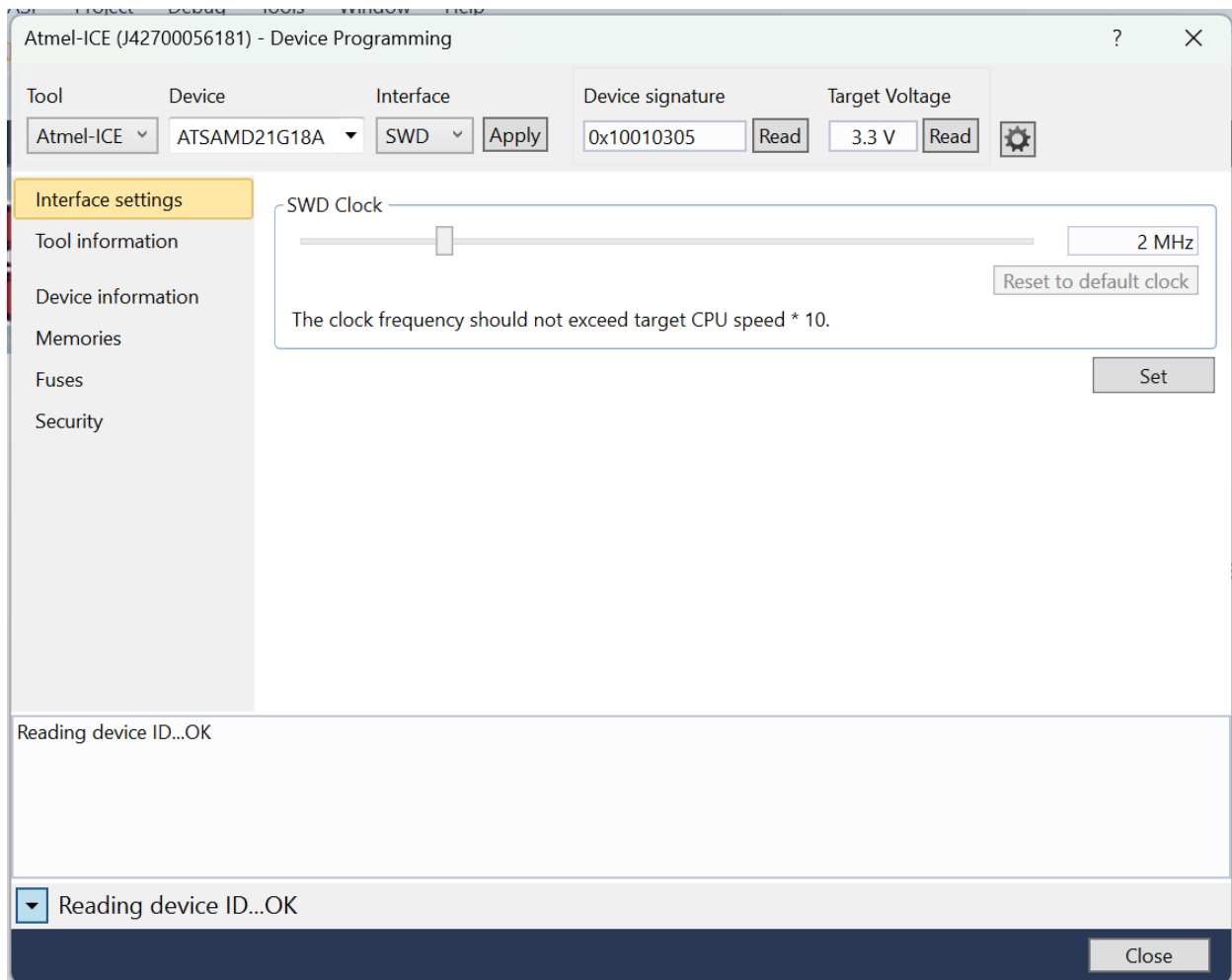
Open Microchip studio and select “Device Programming” from the “tools” menu as shown below.



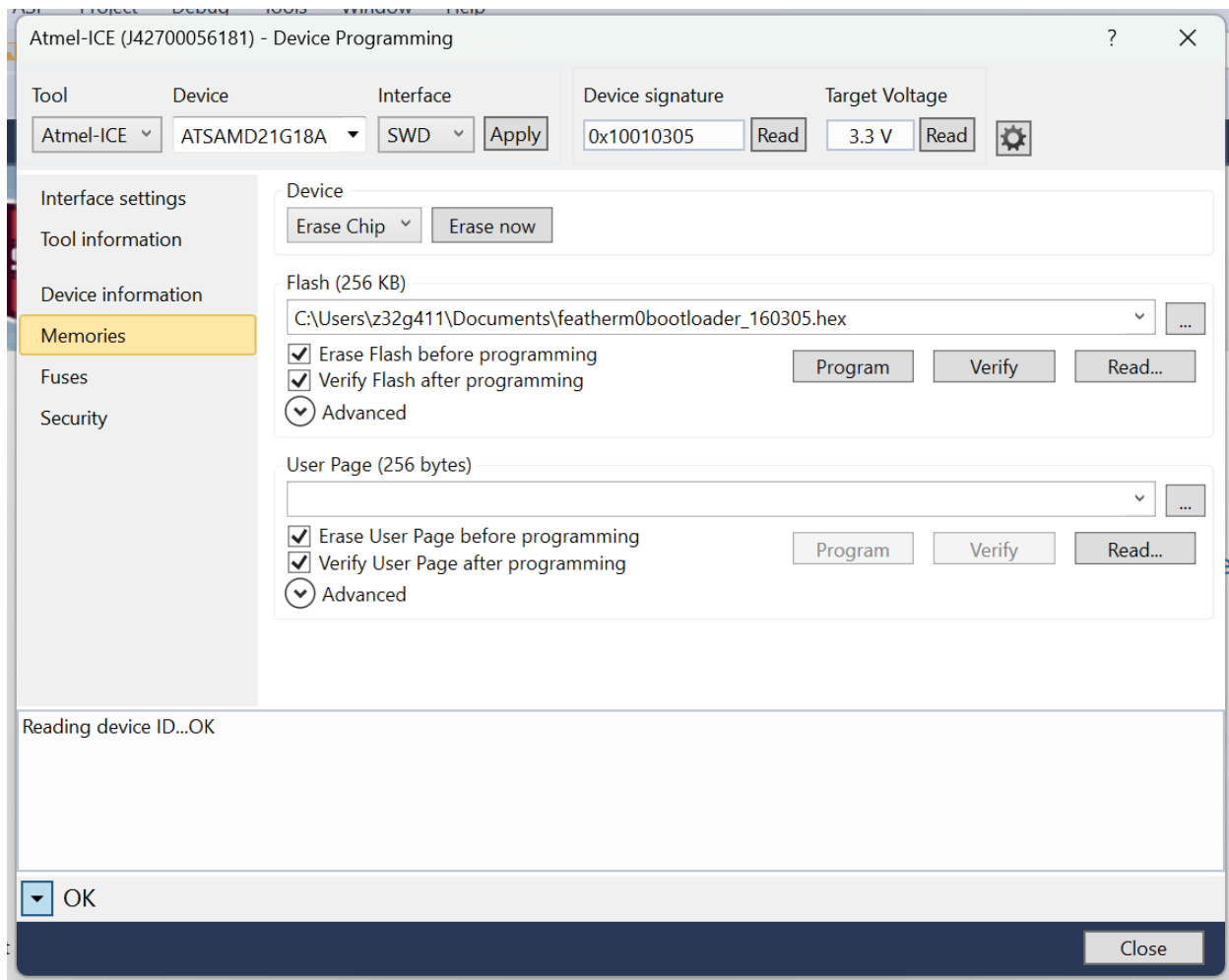
Select the correct programmer (in this case I am using an Atmel ICE) from the tool drop down menu on the left side of the window. Select your microcontroller (ATSAMD21G18A) from the “device” drop down menu. Select “SWD” from the interface menu. Then click “apply”. Once this is done you should see the screen below.



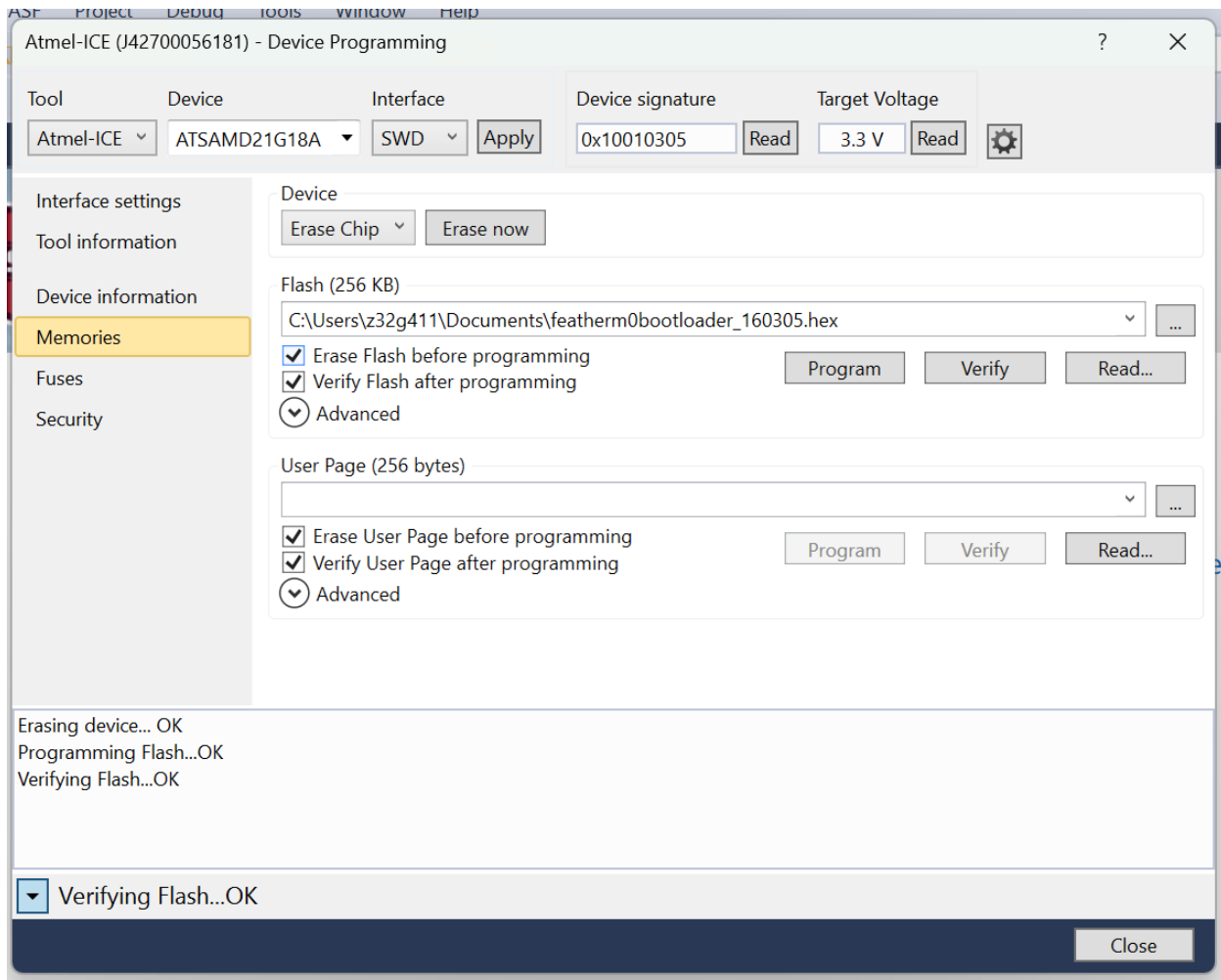
Then click “read” and you should see this screen:



Next, click on the “memories” tab on the left side of the window and then, to the right of the textbox labeled “Flash (256KB)” click the three small “...”. Navigate to where you have downloaded the Adafruit “featherm0bootloader_160305.hex” file. You can download a copy of it [here](#): You should now see something similar to the picture below, although the path to your hex file will be different from mine.



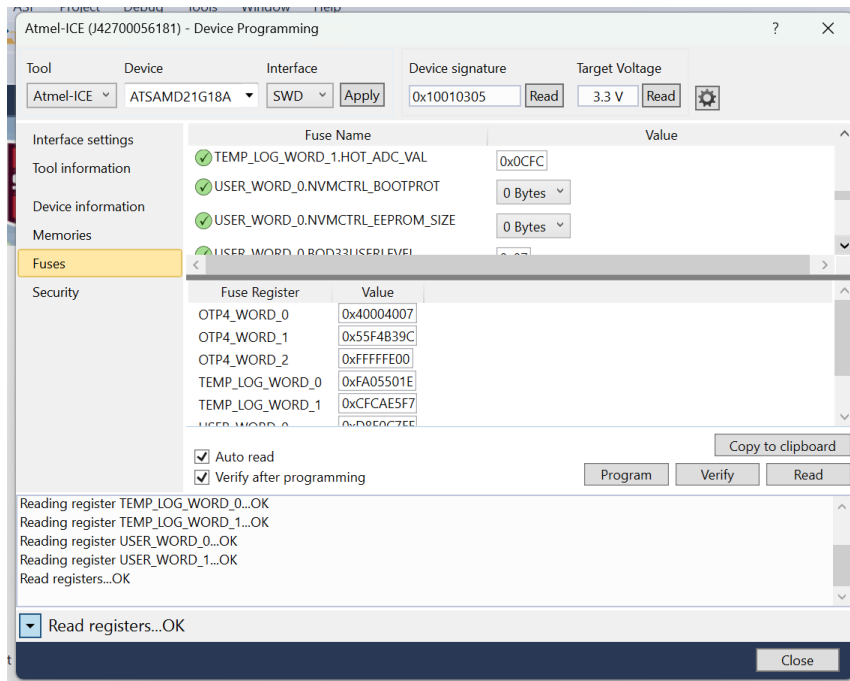
Finally, click on “program”. If all goes well you should see the screen below. If so, you are finished and can now connect your YELLOWSTONE PCB to your computer and program using the Arduino IDE. If you do not have Arduino on your computer you can download the latest version here: <https://www.arduino.cc/en/software/>



If this does not work the most common mistakes are having the SWDIO and SWCLK lines reversed. Also, make sure the board is powered with a battery and that you have the 3.3 and GND lines from the battery going between the programmer and the YELLOWSTONE pcb.

Setting the bootloader fuses (optional)

You can set the bootloader fuses which will prevent accidental overwrite of the bootloader. I typically do not do this as I have found it not to be an issue, however, there is always a chance you do something strange in your code that causes an overwrite of this block of memory. If you wish to prevent an overwrite of the bootloader you can click on the “fuses” tab, then look for “USER_WORD_O_NVMCTRL_BOOTPROT” You will need to scroll through the fuse menu to find this item as it is located a little bit down the list. Then, from the drop down menu next to “USER_WORD_O_NVMCTRL_BOOTPROT” change 0 bytes to 8,192 bytes. Then hit “program”. If you don’t get any errors, you are finished!



Next steps

Once the bootloader has been successfully installed, YELLOWSTONE needs to go through software checks of all basic systems. You will need a copy of the Arduino IDE installed on your computer. You can download the latest version here:

<https://www.arduino.cc/en/software/>

The first test program that you should run on the YELLOWSTONE PCB should be BLINK.

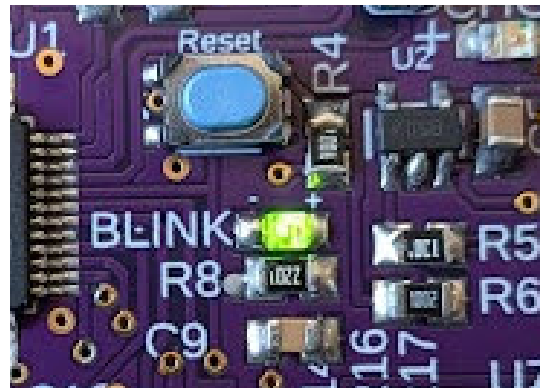
A successful test of BLINK lets us know that the microcontroller is alive and functioning and that the Arduino IDE is able to communicate with the microcontroller and install code.

Hardware Tests and Arduino Code Samples

BLINK

The BLINK sample code comes with the Arduino IDE, however I have also included a copy of it below that you can copy paste into Arduino to test your YELLOWSTONE pcb. You will know that blink is working if the “BLINK” led shown below (green rectangle right

of the word “Blink”) is flashing at a rate of once per second. The color of the BLINK LED may vary from board to board. The typical color is green, however color is irrelevant.



Blink Sample Test Code

```
// the setup function runs once when you press reset or power  
the board  
void setup() {  
    // initialize digital pin LED_BUILTIN as an output.  
    pinMode(LED_BUILTIN, OUTPUT);  
}  
  
// the loop function runs over and over again forever  
void loop() {  
    digitalWrite(LED_BUILTIN, HIGH);    // turn the LED on (HIGH is  
the voltage level)  
    delay(1000);                        // wait for a second  
    digitalWrite(LED_BUILTIN, LOW);     // turn the LED off by  
making the voltage LOW  
    delay(1000);                        // wait for a second  
}
```

SD

To test the SD card functionality, insert a FAT32-formatted microSD card into the card holder on YELLOWSTONE. Most cards 32GB or smaller are already formatted correctly. Larger cards (64GB or more) may need to be reformatted to FAT32 before use.

Upload the following code and open the Serial Monitor. You should see messages indicating whether the SD card initialized successfully, along with file read/write activity.

After running the test, remove the SD card and inspect it on a computer using a microSD card reader. Confirm that a file named test.txt was created and contains the text:

“testing 1, 2, 3.”

SD Card Sample Test Code

```
#include <SPI.h>
#include <SD.h>

File myFile;

void setup() {
  // Open serial communications and wait for port to open:
  Serial.begin(9600);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for native
    USB port only
  }

  Serial.print("Initializing SD card...");

  if (!SD.begin(4)) {
    Serial.println("initialization failed!");
    while (1);
  }
  Serial.println("initialization done.");

  // open the file. note that only one file can be open at
  a time,
  // so you have to close this one before opening another.
  myFile = SD.open("test.txt", FILE_WRITE);

  // if the file opened okay, write to it:
```

```

if (myFile) {
    Serial.print("Writing to test.txt...");
    myFile.println("testing 1, 2, 3.");
    // close the file:
    myFile.close();
    Serial.println("done.");
} else {
    // if the file didn't open, print an error:
    Serial.println("error opening test.txt");
}

// re-open the file for reading:
myFile = SD.open("test.txt");
if (myFile) {
    Serial.println("test.txt:");

    // read from the file until there's nothing else in it:
    while (myFile.available()) {
        Serial.write(myFile.read());
    }
    // close the file:
    myFile.close();
} else {
    // if the file didn't open, print an error:
    Serial.println("error opening test.txt");
}
}

void loop() {
    // nothing happens after setup
}

```

Pressure Sensor (Altitude)



To test the pressure sensor, upload the provided code and open the Serial Monitor set to 115200 baud. Note that you will need to install the MS5X.h library. If you need assistance installing Arduino libraries, refer to the Arduino IDE documentation or visit: <https://docs.arduino.cc/software/ide-v1/tutorials/installing-libraries/>

Upon startup, the system will attempt to detect the sensor on the I²C bus using the common addresses (0x76 and 0x77). If the sensor is detected, the Serial Monitor will display a confirmation message followed by continuous readings of pressure (in Pascals) and temperature (in °C). These values should update once per second.

If the sensor is not detected, an error message will be displayed. This typically indicates an issue with wiring, power, or the I²C connection. Verify that the SDA and SCL lines are connected correctly and that the sensor is receiving proper power.

Successful output confirms that the sensor is communicating correctly and is ready for use in data collection applications. Be sure to use the sample code provided here, as the default example included with the MS5X.h library may report a connection error. This occurs because the YELLOWSTONE board uses a different I²C address than the one assumed in the library's default example.

Pressure Sensor Sample Test Code

```
#include <Wire.h>
#include <MS5x.h>
#include <math.h>

MS5x barometer(&Wire);

unsigned long lastPrint = 0;
const unsigned long printInterval = 1000;
```

```

bool tryBarometerAddress(uint8_t addr) {
    barometer.setI2CAddr(addr);
    int status = barometer.connect();

    if (status == 0) {
        Serial.print(F("Barometer found at I2C address 0x"));
        Serial.println(addr, HEX);
        return true;
    }

    Serial.print(F("No barometer at 0x"));
    Serial.print(addr, HEX);
    Serial.print(F(" connect code = "));
    Serial.println(status);
    return false;
}

void setup() {
    Serial.begin(115200);
    delay(2000);

    Wire.begin();
    delay(100);

    bool connected = false;

    // Try the two common MS5607/MS5611 I2C addresses
    if (tryBarometerAddress(0x76)) {
        connected = true;
    }
    else if (tryBarometerAddress(0x77)) {
        connected = true;
    }

    while (!connected) {
        Serial.println(F("Could not find barometer at 0x76 or 0x77.
Retrying..."));
        delay(1000);
    }
}

```

```

    if (tryBarometerAddress(0x76)) {
        connected = true;
    }
    else if (tryBarometerAddress(0x77)) {
        connected = true;
    }
}

Serial.println(F("Connected to sensor"));
}

void loop() {
    barometer.checkUpdates();

    if (barometer.isReady()) {
        double temperatureC = barometer.GetTemp();
        double pressurePa = barometer.GetPres();

        if (millis() - lastPrint >= printInterval) {
            lastPrint = millis();

            double altitudeM = 44330.0 * (1.0 - pow(pressurePa / 101325.0,
0.1903));

            Serial.print(F("Temp C: "));
            Serial.println(temperatureC, 2);

            Serial.print(F("Pressure Pa: "));
            Serial.println(pressurePa, 2);

            Serial.print(F("Altitude m: "));
            Serial.println(altitudeM, 2);

            // CSV output for logging / LoRa formatting later
            Serial.print(F("CSV: "));
            Serial.print(temperatureC, 2);
            Serial.print(",");
            Serial.print(pressurePa, 2);
            Serial.print(",");
            Serial.println(altitudeM, 2);

```

```

        Serial.println();
    }
}
}

```

GPS



You will need to install the Sparkfun GPS library “SparkFun_u-blox_GNSS_Arduino_Library.h”. You can download it from their github here: https://github.com/sparkfun/SparkFun_u-blox_GNSS_Arduino_Library

To test the GPS module, upload the provided code and open the Serial Monitor (set to 115200 baud). After initialization, the system will attempt to communicate with the GNSS module over the I²C bus. If the module is detected, the Serial Monitor will begin displaying position data once per second, including latitude, longitude, altitude, and the number of satellites in view (SIV). Latitude and longitude are reported in degrees multiplied by 10^{-7} , and altitude is reported in millimeters. **For best results, ensure the GPS antenna has a clear view of the sky, as the module requires satellite signals to calculate an accurate position. Generally, you cannot get a GPS lock indoors.** If the module is not detected, an error message will be displayed—this typically indicates a wiring issue or lack of power. Successful output confirms that the GPS module is properly communicating and ready for use in location-based applications. Note that it can take several minutes to accrue a GPS signal, so be patient.

GPS Sample Test Code

```

#include <Wire.h> //Needed for I2C to GNSS

#include <SparkFun_u-blox_GNSS_Arduino_Library.h>
//http://librarymanager/All#SparkFun_u-blox_GNSS

```

```

SFE_UBLOX_GNSS myGNSS;

long lastTime = 0; //Simple local timer. Limits amount if I2C
traffic to u-blox module.

void setup()
{
    Serial.begin(115200);
    while (!Serial); //Wait for user to open terminal
    Serial.println("SparkFun u-blox Example");

    Wire.begin();

    //myGNSS.enableDebugging(); // Uncomment this line to enable
    helpful debug messages on Serial

    if (myGNSS.begin() == false) //Connect to the u-blox module using
    Wire port
    {
        Serial.println(F("u-blox GNSS not detected at default I2C
address. Please check wiring. Freezing.));
        while (1);
    }

    myGNSS.setI2COutput(COM_TYPE_UBX); //Set the I2C port to output
    UBX only (turn off NMEA noise)
    myGNSS.saveConfigSelective(VAL_CFG_SUBSEC_IOPORT); //Save (only)
    the communications port settings to flash and BBR
}

void loop()
{
    //Query module only every second. Doing it more often will just
    cause I2C traffic.
    //The module only responds when a new position is available
    if (millis() - lastTime > 1000)
    {
        lastTime = millis(); //Update the timer

        long latitude = myGNSS.getLatitude();

```

```

Serial.print(F("Lat: "));
Serial.print(latitude);

long longitude = myGNSS.getLongitude();
Serial.print(F(" Long: "));
Serial.print(longitude);
Serial.print(F(" (degrees * 10^-7)"));

long altitude = myGNSS.getAltitude();
Serial.print(F(" Alt: "));
Serial.print(altitude);
Serial.print(F(" (mm)"));

byte SIV = myGNSS.getSIV();
Serial.print(F(" SIV: "));
Serial.print(SIV);

Serial.println();
}
}

```

Gyro

Gyro Sample Test Code

<insert Gyro Test code Here>

LoRa

To test the LoRa module you will need to install the Radiohead (RH_RF95.h) Library. You can download the latest from their github here:

<https://github.com/hallard/RadioHead>

You will need two YELLOWSTONE boards to test LoRa and an antenna connected to each board.

LoRa Test Transmit code (Tx)

This should go on the “airborne unit”.

```
#include <SPI.h>
#include <RH_RF95.h>

// Change these to match your wiring
#define RFM95_CS    8
#define RFM95_INT   3
#define RFM95_RST   4

// Choose the correct frequency for your module/region
#define RF95_FREQ 915.0

RH_RF95 rf95(RFM95_CS, RFM95_INT);

void setup()
{
    pinMode(LED_BUILTIN, OUTPUT);

    Serial.begin(115200);
    while (!Serial) ;    // remove if not needed on your board

    // Reset the LoRa module
    pinMode(RFM95_RST, OUTPUT);
    digitalWrite(RFM95_RST, HIGH);
    delay(10);
    digitalWrite(RFM95_RST, LOW);
    delay(10);
    digitalWrite(RFM95_RST, HIGH);
    delay(10);

    Serial.println("Starting LoRa transmitter...");

    if (!rf95.init())
```

```

{
    Serial.println("LoRa init failed");
    while (1);
}

Serial.println("LoRa init OK");

if (!rf95.setFrequency(RF95_FREQ))
{
    Serial.println("setFrequency failed");
    while (1);
}

// 13 to 23 dBm depending on hardware
rf95.setTxPower(20, false);

Serial.print("Frequency set to: ");
Serial.println(RF95_FREQ);
}

void loop()
{
    const char *message = "Hello from LoRa!";

    Serial.print("Sending: ");
    Serial.println(message);

    rf95.send((uint8_t*)message, strlen(message));
    rf95.waitPacketSent();

    digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));

    delay(1000);
}

```

LoRa Test Receive code (Rx)

This should go on the “Ground Station unit”.

```
#include <SPI.h>
#include <RH_RF95.h>

#define RFM95_CS    8
#define RFM95_INT    3
#define RFM95_RST    4

#define RF95_FREQ 915.0

RH_RF95 rf95(RFM95_CS, RFM95_INT);

void setup()
{
    Serial.begin(115200);
    while (!Serial) ;

    pinMode(RFM95_RST, OUTPUT);
    digitalWrite(RFM95_RST, HIGH);
    delay(10);
    digitalWrite(RFM95_RST, LOW);
    delay(10);
    digitalWrite(RFM95_RST, HIGH);
    delay(10);

    Serial.println("Starting LoRa receiver...");

    if (!rf95.init())
    {
        Serial.println("LoRa init failed");
        while (1);
    }
}
```

```

if (!rf95.setFrequency(RF95_FREQ))
{
    Serial.println("setFrequency failed");
    while (1);
}

Serial.println("Receiver ready");
}

void loop()
{
    if (rf95.available())
    {
        uint8_t buf[RH_RF95_MAX_MESSAGE_LEN];
        uint8_t len = sizeof(buf);

        if (rf95.recv(buf, &len))
        {
            Serial.print("Received: ");
            Serial.write(buf, len);
            Serial.println();

            Serial.print("RSSI: ");
            Serial.println(rf95.lastRssi(), DEC);
        }
    }
}

```

Testing the LoRa code is a bit more involved because you need two YELLOWSTONE boards and, ideally, two computers—one for each board. It is possible to complete this test with a single computer, but it can become confusing, so using two computers is recommended whenever possible.

Install the Tx code and its associated libraries on computer 1, and install the Rx code and its associated libraries on computer 2. Open the Serial Monitor on both computers.

You should see data messages being transmitted back and forth between the two YELLOWSTONE boards. If you do, the LoRa hardware is working properly. At that point, you can begin testing antennas and communication range to determine what type of distances you can achieve.

If you do not see data being exchanged, first verify that the correct software is installed on each system, with Tx on one board and Rx on the other. Also confirm that antennas are installed on both YELLOWSTONE PCBs. Finally, try changing the distance between the two boards. In some cases, having them too close together can also cause communication problems.

Setting up YELLOWSTONE for flight operations

!!!NOTE!!!: Make sure an antenna is connected to the YELLOWSTONE PCB before powering on. **If the board attempts to transmit without an antenna attached the LoRa radio can get damaged!**

Ground Side Setup

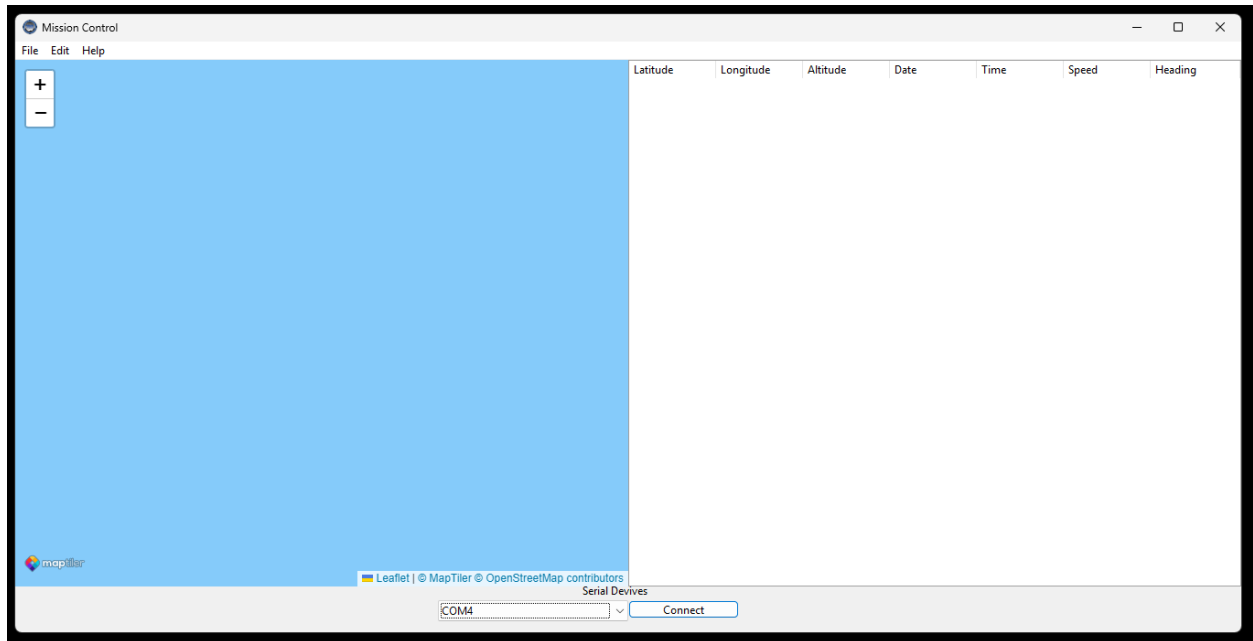
The YELLOWSTONE PCB is designed to work as an airborne device or a ground station. The same PCB can serve either role with a simple swap of firmware. To begin, start with the ground station PCB and install the latest ground station code:

<insert link to current version of ground station code here>

Mission Control PC Software

Download the latest version of the PC mission control software here. Versions are available to Windows, MAC OS, and Raspberry Pi (you only need a basic computer to run the mission control software). This software is primarily developed for Windows, therefore the Mac and Linux versions are often not as up to date as the Windows version.

[Software Download](#)

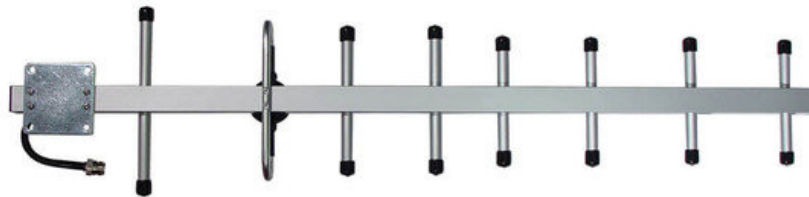


Airborne Side Setup

Install the latest version of the YELLOWSTONE airborne code

<insert link to current version of airborne code here>

Connect your groundstation YELLOWSTONE PCB to the computer running the mission control software, via a micro USB cable. Make sure that an antenna is connected to the YELLOWSTONE pcb (A Yagi antenna is recommended for maximum range for hand pointing).



Yagi Antenna

Make sure that your airborne YELLOWSTONE pcb has an antenna connected.

Power on both the airborne and groundstation YELLOWSTONE boards if they are not already.

Open the mission control software and select your YELLOWSTONE board from the dropdown menu as shown below:

<insert image>

You should now see your aircraft track on the screen assuming the airborne unit has a good GPS lock and is transmitting data.